

Rappresentazione Fixed-Point e Errore di Quantizzazione - Esercitazione 8

L'elaborazione dei segnali è quasi sempre fatta in digitale (conviene) seppur i segnali (uscita ed entrata) sono in analogico:

$$x(t) \rightarrow \text{ADC} \rightarrow x[n] \rightarrow \text{Processing} \rightarrow y[n] \rightarrow \text{DAC} \rightarrow y(t)$$

Bisogna però considerare che quando si fa l'elaborazione digitale di un segnale, si ha una perdita di informazioni, per cui il segnale di uscita $y(t)$ viene identificato come $\tilde{y}(t)$ per far fronte a questo fatto.

L'elaborazione digitale può essere hardware o software:

- **Hardware (circuiti):** FPGA (Field Programmable Gate Array), ovvero una scheda con gate elementari programmabile tramite VHDL, oppure ASIC (Application Specific Integrated Circuit).
- **Software:** Si programma un microprocessore, solitamente in C, DSP (Digital Signal Processor), oppure MIC (micro-controllori) utili perché architettati per svolgere ripetutamente la stessa task velocemente.

Nel caso hardware l'elaborazione deve avvenire in un certo periodo di campionamento, man mano che aumento F_s ho sempre meno tempo per svolgere l'elaborazione (deve essere più veloce), l'hardware viene scelto quando ho bisogno di F_s grandi, infatti oltre i 100 MHz si fa fatica senza hardware.

Nel caso software la velocità dell'elaborazione è più bassa (di circa un fattore 100 rispetto l'hardware), ma il vantaggio principale è l'intercambiabilità, infatti per cambiare l'elaborazione basta caricare il software modificato sul microprocessore, invece per l'hardware dovrei rifare tutto da capo.

Il generico x_n è una generica parola di b bit (solitamente 16), in binario per rappresentare un numero decimale ho due opzioni:

- **Virgola Mobile** con il tipo float/double
 - **Virgola Fissa** con il tipo int
- Viene sempre usata la seconda opzione ovvero la virgola fissa.

Un modo semplice per ricordare la notazione la virgola mobile è pensare alla notazione scientifica 10^n ma cambiare la base in 2^n , infatti basta trovare il primo 1 e spostare la virgola:

$$1234.5 = 1.2345 \cdot 10^3$$

da cui:

$$1101.1 = 1.1011 \cdot 2^3$$

dove se la prima cifra è 1 allora il numero è *negativo*, comunque a noi non interessa.

L'utilizzo della virgola fissa è principalmente dovuto ad una questione di efficienza, infatti per fare moltiplicazioni tra interi basta un solo Clock di CPU, invece per i floating point potrebbero servire 5-6 Clock di CPU.

Quindi si scambia la precisione molto alta dei floating point, per il costo computazionale minore degli int (oltretutto alcuni processori non supportano la virgola mobile, l'ALU per esempio).

Per la rappresentazione in virgola fissa si *decide a priori* quanti bit si usano per la parte intera del numero decimale e quanti bit per la parte decimale del numero decimale, ad esempio usando il formato Q4.12 si trova la rappresentazione in virgola fissa di:

$$3.157 \rightarrow Q4.12 \rightarrow 12931$$

L'algoritmo da seguire è molto semplice infatti basta: prendere il numero decimale, moltiplicarlo per 2^n e troncare la parte decimale:

$$11.451 \rightarrow Q4.12 \rightarrow 11.451 \cdot 2^{12} \text{ (m = 4, n = 12)} \rightarrow 46903,296 \rightarrow 46903$$

per ritornare alla rappresentazione decimale basterà dividere per 2^n :

$$46903/2^{12} \rightarrow 11.45092773$$

Si avrà quindi un piccolo errore chiamato errore di quantizzazione, dovuto al fatto che la rappresentazione in virgola fissa ha una risoluzione minore rispetto alla virgola mobile

In C il formato usato per int a 16 bit sarebbe lo `short int`, però a seconda del compilatore ci potrebbero essere delle ambiguità, perciò per essere sicuri si userà il formato `int16_t` incluso nella libreria `stdint.h`.

Parte 1-2

Si veda il codice sorgente `exercise1.c` e i plot `exercise1_xtilde_plot.pdf` e `exercise1_error_plot.pdf`.

Homework

Si veda il codice sorgente `homework.c` e i plot `homework_xtilde_plot.pdf` e `homework_error_plot.pdf`.